



Should You Go Beyond Relational Databases?



Treehouse

December 16, 2024 - 7 min read

[Learn](#)

Last Updated on May 12, 2026 by [Kari Brooks](#)

Relational databases, such as MySQL, PostgreSQL, and various commercial products, have been foundational to software development for decades. They're reliable, well-documented, and supported by a vast ecosystem of tools and libraries. But as technology evolves and the demands on our systems change, we find ourselves asking: Is the relational model still the best choice for all applications?

[Contents](#) [\[show\]](#)

Signs You May Be Pushing Relational Databases to Their Limits

Before making any drastic changes, take a step back and assess whether your current relational database is still serving your needs. Many applications can run perfectly fine on relational databases for years, but there are specific signs that you might be outgrowing this model.

Structural Symptoms

1. **Sparse Tables:** Do you have tables with a large number of columns, most of which are irrelevant for a given row?
2. **Attribute Tables:** Are you using tables to store (foreign key, attribute name, attribute value) triples, resulting in complex joins?
3. **Serialized Data:** Have you started storing structured data as JSON, XML, or YAML in a single column because your schema can't accommodate it?
4. **Complex Relationships:** Does your schema include numerous many-to-many join tables or tree-like structures with recursive foreign keys?
5. **Frequent Schema Changes:** Do you often need to modify your schema to represent new types of data?

Scalability Symptoms

[Sign In](#) [Free Trial](#)

2. **Data volume.** Is your dataset too large to be stored effectively on a single server?

3. **Performance Issues:** Are background batch processes or analytical queries slowing down your transactional workloads?

If you're encountering these issues, it's time to consider alternative database models.

Non-Relational Databases: An Overview

The past decade has seen a proliferation of non-relational (or NoSQL) databases designed to address the limitations of traditional relational systems. Here are some common types and their use cases:

Key-Value Stores

Key-value stores operate much like hash tables: you insert and retrieve values using a unique key. These databases excel at handling simple, fast lookups and are often distributed to support massive scaling needs.

- **Examples:** Redis, DynamoDB, Aerospike
- **Best For:** Caching, session storage, user preferences, and other low-latency, key-based retrievals.
- **Key Features:** Low latency, high throughput, and the ability to scale horizontally across clusters of servers.

Document Databases

Document databases are designed to handle semi-structured data, often using formats like JSON or BSON. They allow for flexible schemas, making them ideal for

- **Best For:** Applications such as content management systems, product catalogs, and user profiles.
- **Key Features:** Document databases support querying and indexing fields within documents, enabling powerful filtering and aggregation without the need for custom indexing logic.

Column-Family Stores

- **Examples:** Apache Cassandra, HBase
- **Best For:** Large-scale analytical workloads and high-write-throughput scenarios, such as event logging or time-series data.

Graph Databases

Graph databases are designed for applications with highly interconnected data. Unlike relational databases, which struggle with variable-length chains of relationships, graph databases are optimized for traversing relationships efficiently.

- **Examples:** Neo4j, Amazon Neptune, ArangoDB
- **Best For:** Social networks, recommendation engines, fraud detection, and knowledge graphs.
- **Key Features:**
 - Handle complex queries involving transitive relationships, such as friend-of-a-friend queries.
 - Efficiently manage many-to-many relationships and hierarchical data.
 - Use query languages like Cypher or Gremlin for graph traversal.

Bigtable-Inspired Databases

- **Examples:** Google Bigtable, Apache Cassandra, HBase
- **Best For:** IoT data, time-series data, and analytical processing.
- **Key Advantage:** Schema-less design allows for dynamic data models, making it suitable for rapidly evolving datasets.

Distributed Key-Value Stores

Distributed key-value stores extend the simplicity of key-value stores to handle vast amounts of data across clusters of machines. These systems provide horizontal scalability and fault tolerance, making them ideal for high-demand applications.

- **Examples:** Amazon DynamoDB, Azure Cosmos DB, ScyllaDB
- **Best For:** Large-scale applications requiring low-latency access to massive datasets, such as e-commerce, gaming, and IoT.
- **Key Features:**
 - Automatic data partitioning and replication.
 - Transparent sharding to distribute data and workload.
 - Eventual consistency models with optional strict consistency for specific operations.

When selecting a distributed key-value store, it's important to balance low latency (for request-response cycles) and high throughput (for batch processing). Be aware of the trade-offs outlined in Brewer's CAP Theorem: you can prioritize only two out of consistency, availability, and partition tolerance.

MapReduce and Batch Processing

[Sign In](#)[Free Trial](#)

infrastructure. Tools like [Integrate.io](#) provide low-code solutions for building ETL and ELT data pipelines that make it easier to transform and move data across your database ecosystem.

- **Examples:** Apache Hadoop, Apache Spark
- **Best For:** Background data processing, such as log analysis, ETL pipelines, and machine learning workflows.
- **Key Features:**
 - Divide-and-conquer approach to data processing.
 - Integrates well with distributed storage systems like HDFS or Amazon S3.

CouchDB and MongoDB also incorporate MapReduce-like functionality on a smaller scale, allowing for local aggregation and filtering within the database itself.

Free Trial

Try it for 7 days, then \$25 a month

Email

Get Started





something new? Consider the following factors:

1. Data Structure

- If your data fits well into rows and columns, relational databases remain a solid choice.
- If your data is hierarchical, sparse, or highly interconnected, explore document or graph databases.

2. Scalability Requirements

- Relational databases can scale vertically (adding more resources to a single server) or horizontally (via sharding), but these solutions have limits.
- For massive scalability, consider distributed systems like Cassandra or DynamoDB.

3. Query Patterns

- For simple key-based lookups, key-value stores excel.
- For complex relationships, graph databases shine.
- For flexible queries on semi-structured data, document databases are ideal.

4. Operational Complexity

- Relational databases have mature ecosystems and are generally easier to manage. If you need governed API access to your database for enterprise applications or LLMs, [DreamFactory](#) provides a self-hosted platform with role-based access control and identity passthrough.
- Non-relational databases often require specialized knowledge and operational expertise.

[Sign In](#)[Free Trial](#)

-
- If hiring or developer onboarding is a concern, consider sticking with technology your team already knows.
-

Trends in Database Technology

Multi-Model Databases

Many modern databases, such as PostgreSQL and ArangoDB, now support multiple data models (e.g., relational, document, graph) within the same system. This hybrid approach can provide flexibility without the need to adopt entirely new technologies.

Serverless Databases

Cloud providers like AWS and Azure offer serverless database solutions that automatically scale and manage infrastructure for you, reducing operational overhead.

AI-Driven Query Optimization

Emerging databases are leveraging AI to optimize query performance dynamically, further improving efficiency.

- 1.

[Sign In](#) [Free Trial](#)

Relational databases are not obsolete; they are robust, mature, and versatile.

However, they are not a one-size-fits-all solution. When your application's needs evolve—whether due to scalability, data structure, or performance requirements—it's worth exploring other options. By understanding your specific use case and the strengths of different database technologies, you can make an informed decision that ensures your application's success.

Remember: The best database is the one that meets your current and future needs while balancing complexity, cost, and performance.

Ready to learn more? Explore advanced database technologies with hands-on courses at Treehouse. [Start Your Free Trial Today!](#)

Like this article?

If you enjoyed this article, check out another one:

[The AI Multiverse: Exploring the Subfields of AI](#)

code

[Sign In](#) [Free Trial](#)

39 Responses to “Should You Go Beyond Relational Databases?”

SQL Lion on August 3, 2010 at 11:03 am said:

Very Nice post, quiet informative.

Database designing problems being the buzzwords these days, one of the most common designing mistakes is the “Poor Planning / Architecture” of the database or mart. Follow the link to know more...

<http://www.sqllion.com/2010/08/database-design-and-modeling-i/>

[Log in to Reply](#)

Alex Popescu on August 1, 2009 at 9:45 pm said:

1. Very nice post! A while back I've started a comparison of “alternative” data storage solution. It's far from being complete, I think it might offer some quite details to newcomers

(<http://themindstorms.blogspot.com/2009/05/quick-reference-to-alternative-data.html>).

2. I've been using BDB as a storage implementation of Jackrabbit for 4 years. It was couple of days ago (in fact it happened exactly on Jul. 30th, the date of the last comment mentioning failures of BDB) that this database got corrupted.

3. There are a couple of things about all these solution that concern me and I'm referring here to the fact that most of them have chosen to implement custom protocols and connection APIs. While I may find some reasons for this (f.e. most of them have been initially developed to solve an internal problem and open sourced afterwards), it is quite strange to see that they lock you in. Basically, before starting to use any of these solution you'll have to spend a lot more time to figure out if the solution you are picking will fit all your needs, as later migration will be extremely difficult.

I hope there will be a time when people involved will sit down and figure out some common protocols and APIs. And I hope this will happen sooner than later, as the more we postpone this step the more difficult will be to ask those already using to change their implementations.

[Log in to Reply](#)

Jim McCoy on July 30, 2009 at 5:10 am said:

Glenn: While Berkeley DB is good tech, I think that a lot of us got burned by it in the past (three catastrophic loss of entire db situations and two others that required several days to recover from in my

[Sign In](#) [Free Trial](#)

Glenn Wiens on [July 13, 2009 at 11:09 pm](#) said:

Coming to the end of the comment section, I was disappointed to see no response to a question raised more than once (and one that rolls around in my mind as well): What about Berkeley DB? It has a long-established history and is being used by some of the busier websites to assist in making their RDBMS systems fast enough to keep up with web hits.

Would BDB be considered a NoSQL candidate? Or is the direction of NoSQL just new shinier things?

[Log in to Reply](#)

Rob Tweed on [July 3, 2009 at 7:17 am](#) said:

Also check out M/DB which is an open source clone of SimpleDB, and another from the same stable: M/DB:X which is an HTTP-interfaced lightweight Native XML Database. Indeed M/DB:X is soon to become even more interesting with a new JSON/XML mapping capability built in: input a JSON string and it will be stored as an equivalent XML DOM that can be modified and manipulated via DOM API methods and/or searched using XPath....and then returned as JSON.

[Log in to Reply](#)

Michael_Marth on [July 2, 2009 at 10:00 am](#) said:

Very good article. I like that you stress the need to fit the actual data to the model.

One small correction: Jackrabbit can deal with transactions (see <http://jackrabbit.apache.org/frequently-asked-que...> In the upcoming JSR-283 specification (for which Jackrabbit is the reference implementation and which it partially implements already) query joins are also supported.

[Log in to Reply](#)

Michael_Marth on [July 2, 2009 at 10:00 am](#) said:

Very good article. I like that you stress the need to fit the actual data to the model.

One small correction: Jackrabbit can deal with transactions (see <http://jackrabbit.apache.org/frequently-asked-que...> In the upcoming JSR-283 specification (for which Jackrabbit is the reference implementation and which it partially implements already) query joins are also supported.

[Sign In](#) [Free Trial](#)

Jonathan Gray on June 26, 2009 at 4:56 pm said:

There's another reason small companies are trying to build on scalable technologies early.

It's not only that they want to be prepared for an increase in customers, but their datasets are very large from day one (and growing the dataset 100X could be a matter of flipping a switch).

Many of these companies are mining data, web crawling, etc... It's the original and canonical use case for BigTable, though it's now used for a wide variety of applications.

[Log in to Reply](#)

Drakanor on June 26, 2009 at 3:27 pm said:

I'm still waiting for a database natively supporting the associative data model. To me it's the best solution for 90% of all database based application use cases. Right now, we simulate it with PostgreSQL by putting in an additional "relational-to-associative" layer which of course may raise performance issues in some cases.

[Log in to Reply](#)

Julian Browne on June 26, 2009 at 5:09 am said:

A very good and unbiased round up.

The problem with scalability these days it that the coolness factor is making people want to play at being Amazon or Google so there's a tendency to jump right into solving imaginary scale issues when, as you say, it's a "very remote problem on most projects" and it's also very hard so the end result is often unnecessary complexity, which perversely doesn't then scale.

It ain't like we haven't got enough on our respective plates providing high-quality, extensible, useful, reliable, software in the first place.

[Log in to Reply](#)

John Turnbull on June 25, 2009 at 10:27 pm said:

So many new choices. Here's another: EMC's xDB – a native xml DB with a development solution built around it.

[Log in to Reply](#)

Jerry Silver on August 3, 2009 at 11:41 pm said:

[Sign In](#)[Free Trial](#)

Gergely Orosz on June 25, 2009 at 8:39 pm said:

Nice overview. I am missing some file-based closer to object oriented than relational databases e.g. db4o (<http://www.db4o.com/>) or Karvonite (<http://code.msdn.microsoft.com/karvonite>) in the comparison.

Having mentioned open source document-like databases and Jackrabbit I would also add the Sense/Net Content Repository (http://wiki.sensenet.hu/index.php?title=SenseNet_... to the list which is similar to Jackrabbit and the JSR 170 in the Java world.

[Log in to Reply](#)

Gergely Orosz on June 25, 2009 at 8:39 pm said:

Nice overview. I am missing some file-based closer to object oriented than relational databases e.g. db4o (<http://www.db4o.com/>) or Karvonite (<http://code.msdn.microsoft.com/karvonite>) in the comparison.

Having mentioned open source document-like databases and Jackrabbit I would also add the Sense/Net Content Repository (http://wiki.sensenet.hu/index.php?title=SenseNet_... to the list which is similar to Jackrabbit and the JSR 170 in the Java world.

[Log in to Reply](#)

james on June 25, 2009 at 6:45 pm said:

Martin: too busy to approve my comment huh?

Chris: Thanks for the chuckle of the day.

[Log in to Reply](#)

Peter on June 25, 2009 at 5:44 pm said:

I was just going to post about Persevere as well. I like it because of the way you can extend document types – the object model makes sense to the way I handle document based data.

[Log in to Reply](#)

Jags Ramnarayan on June 25, 2009 at 4:43 pm said:

In-memory data grid or fabric might be something you should consider mentioning in your report. They offer distributed key-value stores without losing support for querying or transactions. The products typically get deployed as a middle tier caching solution that can "read-through", "write-through" or "write-

[Sign In](#) [Free Trial](#)

related data) and support moving behavior to where the data resides (map-reduce).

Hopefully, this white paper – <http://community.gemstone.com/display/gemfire/EDF...> provides some insight into the important concepts.

[Log in to Reply](#)

Jags Ramnarayan on June 25, 2009 at 4:43 pm said:

In-memory data grid or fabric might be something you should consider mentioning in your report. They offer distributed key-value stores without losing support for querying or transactions. The products typically get deployed as a middle tier caching solution that can "read-through", "write-through" or "write-behind" to one or more backend databases. They even offer reliable publish-subscribe features so apps can now receive data change notifications enabling scalable "real time push" all the way to browser clients. Finally, one might find how these products can horizontally partition the data (by colocating related data) and support moving behavior to where the data resides (map-reduce).

Hopefully, this white paper – <http://community.gemstone.com/display/gemfire/EDF...> provides some insight into the important concepts.

[Log in to Reply](#)

buck nuggets on June 25, 2009 at 3:41 pm said:

Very nice – this is the first report I've seen that tries to make sense of the options. Right now we've got a lot of people trying to apply lessons learned in one area to another without realizing what the pros & cons really are. Here's two more considerations.

1. data quality – which might not be a huge deal for a large social networking site – where your data may vary slightly based on which server you hit. But it's a killer for finance, reservations, etc. The RDBMS achieves this with both ACID compliance, model consistency and declarative constraints. The last two are as important as the first in ensuring that all data in the database, regardless of which application wrote it, regardless of when it was written (four years ago or today) all meet the current rules within the database.

2. reporting – there are very, very few applications that don't need reporting. One benefit of the RDBMS today is that it can make reporting very easy to deliver. One of the primary reason that object oriented databases failed ten years ago is that they couldn't support reporting. Most applications that used them had to deal with horrible performance, expensive cost to build reports, or copied data out regularly into an RDBMS and had to pay twice the admin, hardware & licensing costs. Most of the new alternatives to the RDBMS suffer from the same limitations.

[Log in to Reply](#)

Kris Zyp on June 25, 2009 at 1:24 pm said:

[Sign In](#) [Free Trial](#)

Shaun Bruner on [June 25, 2009 at 1:13 pm](#) said:

I too wonder why Berkeley DB never makes these lists? Is it because BDB doesn't have built-in server capability? Every time someone mentions key/value databases, I immediately think of the old dbm-style databases, of which BDB is probably the best known member.

[Log in to Reply](#)

james on [June 25, 2009 at 12:45 pm](#) said:

- > Standard SQL cannot query transitive relationships, i.e. variable-length chains of joins
- > which continue until some condition is reached. Graph databases, on the other hand, are
- > optimised precisely for this kind of data. Look out for these symptoms indicating that your
- > data would better fit into a graph model:

The SQL Standard defines syntax for recursive queries (using WITH RECURSIVE), so yes, you can query transitive relationships. Oracle, DB2, MS SQL are among the commercial vendor systems that support them, and the free RDBMSes Firebird and the almost released PostgreSQL 8.4 will have support too. Please take what you read at neo4j with a couple packets of salt (recursive queries aren't slow due to 'increasing number of joins', either).

As for having to add columns to your database to match incoming data, that really indicates poor upfront design. But still, in most modern RDBMSes, adding to (not altering) table schemas is not very expensive, and you can always create another table with a foreign key to the master table (and update your views, because you aren't querying tables directly from your application, right?) Empty columns take from zero to a byte in most RDBMSes, so not really an issue for storage. If you feel you really need a 'schemaless' "design", some RDBMSes have a 'hash' data type which can be indexed on (key, value) (e.g., the 'hstore' extension for PostgreSQL).

Generally the 'need' for non-relational databases has been spurred by the poor quality of RDBMSes such as MySQL, not by limitations of SQL and Relational Databases, or the Relational Model.

[Log in to Reply](#)

Alessio Saltarin on [June 25, 2009 at 12:18 pm](#) said:

Thank you for your article. I agree that RDBMS are at EOL phase (end-of-life). RDBMS engines were born when computer and I/O were slow. Now, this is no more the case. Well serialized, well "hibernated" data, written in a hierarchical fashion (JSON or XML) just do the job in most cases. Also, we won't need SQL anymore: common languages, such as Python, Ruby, Java or C# are enough, and deal with data (collections, dictionaries) in an easier and more effective way.

irf on June 25, 2009 at 10:46 am said:

Are Object Databases no longer cool?



Anyone wanting to be impressed by a database have a look at <http://www.statbank.dk> – I think it's the worlds largest statistical DB.

[Log in to Reply](#)

irf on June 25, 2009 at 10:46 am said:



Anyone wanting to be impressed by a database have a look at <http://www.statbank.dk> – I think it's the worlds largest statistical DB.

[Log in to Reply](#)

Chris Anderson on June 25, 2009 at 7:27 am said:

CouchDB is actually ideal for storing bank account activity (probably the best of the bunch discussed here), as running totals are incredibly cheap to calculate using incremental map reduce. CouchDB can easily give you a consistent snapshot of an account and its total balance using a very simple reduce function. It's one of the examples we use in our training courses because it shows just how good a fit CouchDB is for that sort of application.

[Sign In](#) [Free Trial](#)

Stefan on June 25, 2009 at 6:23 am said:

Very good overview, thank you for that.

I've seen a presentation on column-based databases by Hasso Plattner a few weeks ago. He spoke about enormous performance increases based on huge datasets with real-world enterprise data. That was quite impressive..

[Log in to Reply](#)

James Pearce on June 25, 2009 at 3:42 am said:

Yes, of course it's not X vs Y as mutually exclusive choices.

The people that should be really thinking carefully are those who are using X for a problem better solved by Y, or Y for a problem better solved by X.

In web apps, where relationships are often more vague and data more sparse and distributed, these new store approaches are fantastic.

On the other hand, in certain enterprise applications, an RDBMS may continue to be the ideal platform, come what may. (Would it be ideal for a bank to store your account activity in a CouchDB document?)

Also there are still unique advantages that RDBMS have – for balance, you could mention how these are lacking in the newer alternatives.

(One example: little native support for aggregation – apparently your application must keep running totals of everything as it goes – and put that in a document too. In Google's App Engine, it's even impossible to *count* more than 1,000 rows in a table, would you believe. Imagine running a bank on that.)

So, thanks: your article is fairly balanced, and just on the right side of the naive "new vs old" arguments that have popped up on this topic.

[Log in to Reply](#)

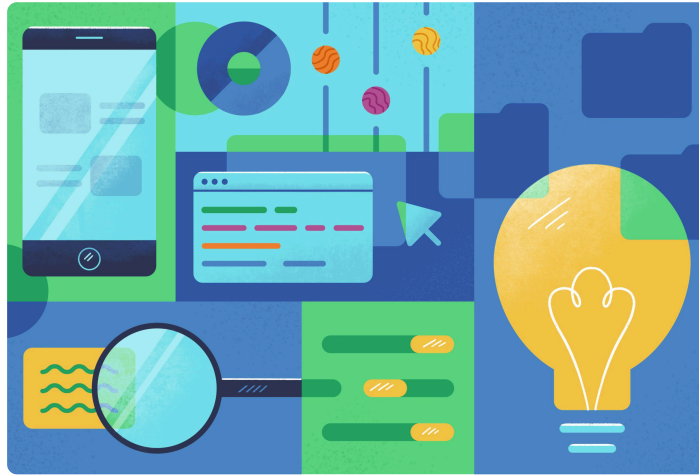
Lucas Vogel on June 25, 2009 at 1:10 am said:

What about the Oracle Berkeley DB product family?

[Log in to Reply](#)

Leave a Reply

You might also like other posts...

[Career Advice](#)

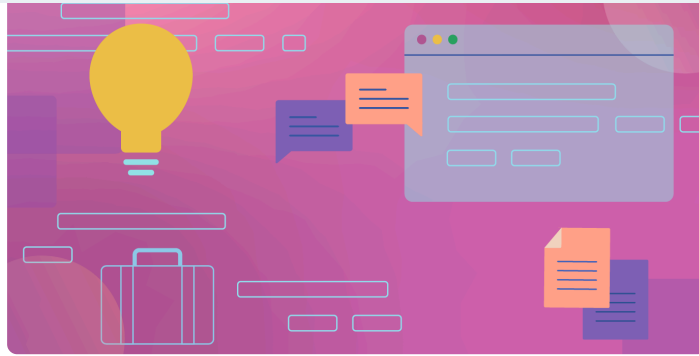
August 7, 2022 • 9 min read

Front-End vs. Back-End: The Complete Guide

If you're new to coding, you may be wondering what the difference is between front-end vs. back-end. Find your answers in our latest Treehouse blog.



Jason Gilmore



Learn

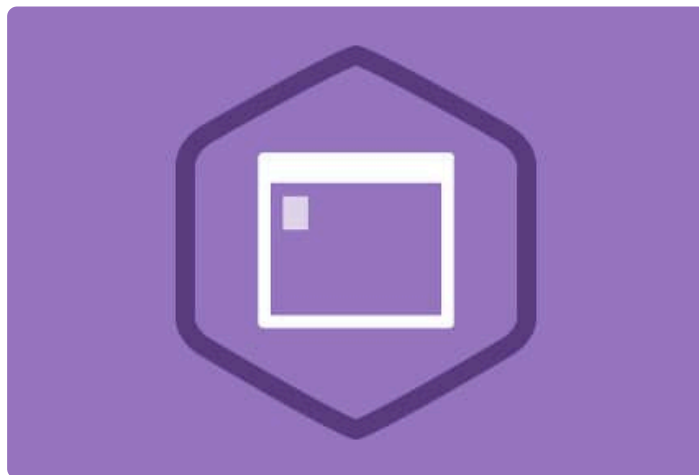
June 7, 2022 • 6 min read

Should I Learn HTML Before Python?

Starting your coding journey and wondering if you should learn HTML before Python? We've got you covered. Keep reading to learn the key differences in our latest blog.



Lindsey Rogerson



Learn

January 18, 2023 • 10 min read

How to Use the Browser Developer Tools Console

The developer console can help you debug your front-end web applications. Learn more

Matt West



Learn to code with Treehouse

Start your 7 day free trial today and get access to hundreds of video courses in web development, design and business!

[Learn more](#)

©2026 Treehouse Island, Inc.

[About](#) • [Careers](#) • [Blog](#) • [Affiliate Program](#) • [Terms](#) • [Privacy](#) • [Contact](#)